

M Ahmed Javed

CU-5276-2024

Assi : 02

Mobile Application Development

Q1.1 — SharedPreferences Limitations

`SharedPreferences` is useful for storing small key-value data (e.g., theme settings, login flags), but it has major limitations in production apps:

1) Limited Data Structure Support

It only stores primitive types:

- `String`
- `int`
- `double`
- `bool`
- `List<String>`

It cannot directly store:

- complex objects
- nested JSON
- relational data

Better Alternative: Hive

Hive can store custom Dart objects using adapters.

Why better?

- Supports object storage
- Faster for structured local data
- No need to manually convert objects into strings

2) Poor Security

Data stored in SharedPreferences is usually stored in plain text.

This creates security risks for:

- tokens
- passwords
- sensitive user information

Better Alternative: Flutter Secure Storage

Use Flutter Secure Storage or platform-specific secure storage.

Why better?

- Uses Android Keystore
- Uses iOS Keychain
- Encrypts sensitive data

3) Not Suitable for Large Data

Reading/writing large datasets can become slow and inefficient.

Example:

- chat history
- large offline notes
- logs

Better Alternative: SQLite or Hive

Use SQLite via sqflite.

Why better?

- Handles large datasets efficiently
- Supports indexing and queries
- Better scalability

Q1.2 — SQLite vs Hive

Suppose we store:

- title
- body
- timestamp
- mood tag

for offline journal entries.

SQLite (sqflite)

Using SQLite through sqflite:

Advantages:

- Supports relationships using foreign keys
- Complex SQL queries
- Filtering/sorting
- Structured schema
- ACID compliance

Example:

```
SELECT * FROM journal
WHERE mood='happy'
ORDER BY timestamp DESC;
```

Disadvantages:

- More boilerplate
- Need SQL knowledge
- Schema migrations required

Hive

Using Hive:

Advantages:

- Very fast key-value storage
- Easy setup

- Pure Dart (no native dependency)
- Great for simple object storage

Disadvantages:

- Weak support for relationships
- Complex filtering is harder
- No SQL querying

Which would I choose?

For a journal app → **Hive**

Why?

Journal entries are usually independent records:

- no complex relationships
- simple CRUD operations
- better developer productivity
- fast local reads/writes

If the app later requires:

- analytics
- complex search
- multiple linked tables

then SQLite becomes the better option.

Q1.3 — Dart Isolates & Storage

Even though file I/O is asynchronous, heavy file operations should still use a separate isolate.

Why?

Async I/O only prevents blocking during waiting time.

But heavy tasks like:

- parsing huge JSON files
- encryption/decryption
- compression
- large file processing

still require CPU work after data is read.

That CPU work runs on the **main isolate**, which can cause:

- UI lag
- dropped frames
- app freezing

Flutter UI runs on a single main isolate, so heavy computation affects responsiveness.

Better Approach: Use Separate Isolate

Move heavy work to another isolate.

APIs used:

- `Isolate.spawn()`
- `compute()` (Flutter helper function)
- `ReceivePort`
- `SendPort`

Example:

```
compute(parseLargeFile, fileData);
```

This keeps UI smooth while heavy processing runs in parallel.

Q1.4 — Data Migration

Suppose:

Version 1 schema:

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
```

```
    name TEXT
)
```

Version 2 adds:

```
email TEXT
```

Step 1: Increase DB Version

```
openDatabase(  
    path,  
    version: 2,  
)
```

Step 2: Use `onUpgrade`

```
onUpgrade: (db, oldVersion, newVersion) async {  
    if (oldVersion < 2) {  
        await db.execute(  
            "ALTER TABLE users ADD COLUMN email TEXT"  
        );  
    }  
}
```

Step 3: Preserve Existing Data

Using `ALTER TABLE` ensures:

- old records remain intact
- app doesn't lose user data

Existing users will simply have:

```
email = NULL
```

until updated.

Step 4: Error Handling

Use transactions and try-catch:

```

onUpgrade: (db, oldVersion, newVersion) async {
  try {
    await db.transaction((txn) async {
      if (oldVersion < 2) {
        await txn.execute(
          "ALTER TABLE users ADD COLUMN email TEXT"
        );
      }
    });
  } catch (e) {
    print("Migration failed: $e");
  }
}

```

Step 5: Backward Compatibility

- New installs → use `onCreate()` with latest schema
- Existing installs → use `onUpgrade()`

Example:

```

onCreate: (db, version) async {
  await db.execute('''
    CREATE TABLE users(
      id INTEGER PRIMARY KEY,
      name TEXT,
      email TEXT
    )
  ''');
}

```

Q2. True / False with Justification

1. SharedPreferences data is automatically encrypted on both Android and iOS.

False

Justification: SharedPreferences stores data in plain text (Android SharedPreferences XML / iOS UserDefaults) and does not provide automatic encryption—use Flutter Secure Storage for sensitive data.

2. Hive TypeAdapters must be registered before any Box is opened.

True

Justification: Custom objects cannot be properly serialized/deserialized unless their TypeAdapters are registered first using `Hive.registerAdapter()` before opening the box in Hive.

3. The path_provider package returns the same directory path on iOS and Android.

False

Justification: path_provider returns platform-specific directories because Android and iOS have different file system structures and sandbox paths.

4. sqflite supports transactions that can be rolled back on failure.

True

Justification: sqflite supports database transactions, and if an operation fails, the transaction can automatically roll back to maintain data consistency.